

SOC/L1 - 7.3. Data Exfiltration Detection

Data Exfiltration Detection

Task 1 - Introduction

Data exfiltration is the unauthorized transfer of sensitive data from a computer or other device. It's a primary objective for attackers who have breached a network. As a SOC analyst, our job is to detect and stop this before sensitive information walks out the door.

Task 2 - Lab Connection

For the practical lab, there are three ways to approach the practical:

- Explore the network traffic in the respective folder in the `data_exfil` directory on the Desktop.
- Perform Log Analysis on the log file in the same folder.
- Examine the logs already ingested into the Splunk instance. Make sure to select All times and use the `index=data_exfil`, as shown below:

The screenshot shows the Splunk Search interface. The search bar contains the query `index=data_exfil`. The search results show 18,117 events. A modal window titled "sourcetype" is open, displaying a table of values for the field "sourcetype". The table lists the following values:

Values	Count	%
sysmon_logs	9,952	54.932%
http_logs	6,165	34.029%
dns_logs	2,000	11.039%

The "dns_logs" entry is highlighted. Below the table, the search results are displayed in a table format, showing the first few lines of the log data.

Task 3 - Data Exfil: Overview, techniques, and Indicators

Answer the questions below

Exfiltrating the data through HTTP comes under which technique? Network-based

Task 4 - Detection: Data Exfil through DNS Tunneling

DNS exfiltration abuses the Domain Name System, a protocol normally allowed through networks, to smuggle bytes encoded inside DNS queries/responses so firewalls and web proxies don't notice. Because DNS is typically allowed and often unfiltered or forwarded to public resolvers, it's attractive for covert channels.

DNS Tunneling: DNS (Domain Name System) translates human-friendly domain names (e.g., `example.com`) into IP addresses and provides other record types (A, AAAA, TXT, MX, CNAME, etc.). Key points:

- DNS queries are ubiquitous: almost every host performs DNS lookups.
- DNS is normally allowed through firewalls and gateways, making it an attractive covert channel.
- DNS uses UDP (mostly) on port 53 for queries and responses; TCP is used for zone transfers or large responses.

Why attackers use DNS for exfiltration:

- Always-on service: DNS lookups are routine and often allowed outbound.
- High cover: queries look like normal requests unless inspected closely.
- Flexible payload: data can be encoded into the subdomain labels or TXT responses.

Indicators of attack: When analysing DNS traffic for possible indicators of data exfiltration, we should look for:

- Many DNS queries are sent to a single external domain, especially with very high counts compared to the baseline.
- Long subdomain labels or unusually long full query names (> 60–100 characters).
- High entropy or Base32/Base64-like patterns in the query name (lots of mixed case letters, digits, `=` signs for base64).

- Rare record types (TXT, NULL) or many large TXT responses.
- Unusual response behavior: frequent NXDOMAIN (if attacker uses exfil-by-query without answering), or TCP/large UDP fragments for DNS.
- Queries at regular intervals (beaconing behaviour).

Detecting through Wireshark

Filter: `dns`

Filter: `dns.flags.response == 0` (We can see some DNS entries with a large query length.)

Filter: `dns && frame.len > 70` (The above result identifies one particular suspicious domain, which is receiving these suspicious-looking DNS requests.)

Investigating with Splunk

Search Query: `index=data_exfil sourcetype=DNS_logs` (This search query will filter show the results matching `sourcetype=DNS_logs`.)

Search Query: `index="data_exfil" sourcetype="DNS_logs" | stats count by src_ip`

Search Query: `index="data_exfil" sourcetype="dns_logs" | stats count by query | sort - count` (The above results, clearly display some odd looking DNS queries with the a large size of the query.)

Search Query: `index="data_exfil" sourcetype="DNS_logs" | where len(query) > 30` Long query names (subdomain encoding), With the following indicators, we were able to identify the data exfiltration attempts through DNS tunneling: A Large number of DNS requests with no response. Large length of the query.

Answer the questions below

What is the suspicious domain receiving the DNS traffic? tunnelcorp.net

run the query: `dns.flags.response == 0`

dns.flags.response == 0						
No.	Time	Source	Destination	Prot	Len	Info
1	0.000000	192.168.1.105	8.8.8.8	DNS	...	Standard query 0x0000 A github.com
2	60.000000	192.168.1.103	8.8.8.8	DNS	...	Standard query 0x0000 A wikipedia.org
3	120.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A g5ft5ma3dtb114q7im4r.tunnelcorp.net
4	180.000000	192.168.1.101	8.8.8.8	DNS	...	Standard query 0x0000 A google.com
5	240.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A 3arwfuze7kctrf0e2ef5.tunnelcorp.net
6	300.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A github.com
7	360.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A google.com
8	420.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A wikipedia.org
9	480.000000	192.168.1.102	8.8.8.8	DNS	...	Standard query 0x0000 A wikipedia.org
10	540.000000	192.168.1.104	8.8.8.8	DNS	...	Standard query 0x0000 A 7qq6jqz7qj0jdp67lesa.tunnelcorp.net

How many suspicious traffic/logs related to dns tunneling were observed? 315

The screenshot shows the Splunk Enterprise interface. At the top, there's a navigation bar with 'splunk>enterprise' and various menu items like 'Messages', 'Settings', 'Activity', 'Help', and 'Find'. Below this is a 'Search & Reporting' section. The main area is titled 'New Search' and contains a search bar with the query: `index="data_exfil" sourcetype="DNS_logs" | where len(query) >30`. Below the search bar, it indicates '315 events (before 7/28/26 11:34:43.000 AM)' and 'No Event Sampling'. There are buttons for 'Save As', 'Create Table View', and 'Close'. At the bottom, there are tabs for 'Events (315)', 'Patterns', 'Statistics', and 'Visualization'.

run the query: `index="data_exfil" sourcetype="DNS_logs" | where len(query) >30`

Which local IP sent the maximum number of suspicious requests? 192.168.1.103

The screenshot shows a field summary for 'src_ip' in Splunk. The field is selected, and the summary shows '5 Values, 100% of events'. Below this, there are tabs for 'Reports', 'Top values', 'Top values by time', and 'Rare values'. The 'Top values' tab is active, showing a table of values and their counts. The table has columns for 'Values', 'Count', and '%'. The data is as follows:

Values	Count	%
192.168.1.103	72	22.857%
192.168.1.105	63	20%
192.168.1.104	62	19.682%
192.168.1.102	60	19.048%
192.168.1.101	58	18.413%

Task 5 - Detection: Data Exfil through FTP

FTP (File Transfer Protocol) is one of the oldest protocols for transferring files between a client and server over a TCP/IP network. Attackers use it to move large amounts of data off a network, sometimes via compromised credentials, misconfigured servers, or ephemeral accounts. Detection relies on a mix of packet inspection (FTP only), server logs, SSH session metadata, and network flow/size/pattern analysis.

How adversaries use FTP for exfiltration

- Use **legitimate FTP servers** (public or misconfigured internal servers) to stage/transfer data.
- Use **compromised credentials** (service accounts, user creds).
- Use **non-standard ports** or tunneling to blend with other traffic.

Indicators of attack

What to look for:

- `USER` and `PASS` commands (cleartext credentials).
- `STOR` (upload) and `RETR` (download) commands: repeated or large transfers.
- Large data connections to unusual external IPs, especially outside business hours.
- Data channel openings on ephemeral ports (PASV) paired with large payloads.

`ftp || ftp-data` This filter will isolate the ftp control traffic.

`ftp.request.command == "USER" || ftp.request.command == "PASS"` From the output, we can look for suspicious usernames or weak passwords.

`ftp contains "STOR"` Look for Anomalies in Filenames or Credentials. Right-click a `packet → Follow → TCP Stream`

`ftp contains "csv"` The above results show a suspicious IP connected as Guest account has transferred some sensitive csv files to a suspicious external IP. (**CSV** stands for **Comma-Separated Values**.)

`ftp && frame.len > 90` Identifying the traffic with a large payload size

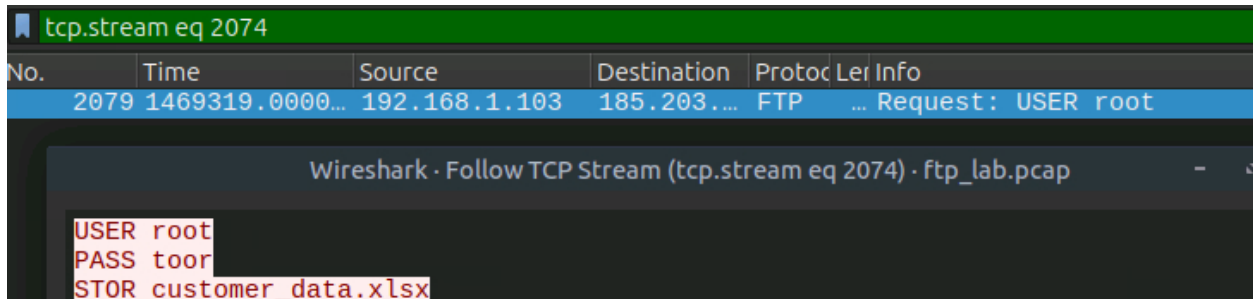
Answer the questions below

| How many connections were observed from the guest account? 5

run the query: `ftp contains "guest"`

| Apply the filter; what is the name of the customer-related file exfiltrated from the root account?

customer_data.xlsx



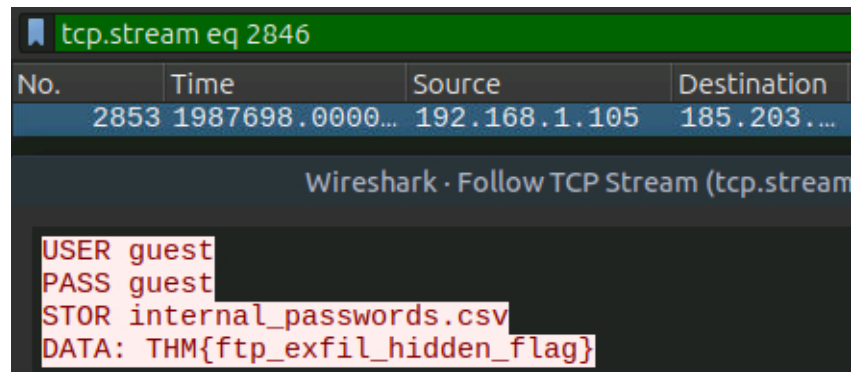
Which internal IP was found to be sending the largest payload to an external IP?

192.168.1.105

What is the flag hidden inside the ftp stream transferring the CSV file to the suspicious IP?

THM{ftp_exfil_hidden_flag}

run the query: ftp contains "csv"



Task 6 - Detection: Data Exfil via HTTP

Data exfiltration via HTTP is when an attacker moves sensitive data out of a target network using HTTP as the transport. HTTP is commonly abused because it blends with normal web traffic, can traverse firewalls and proxies, and can be obfuscated (encoding, encryption, tunneling). This detection task aims to teach SOC analysts how to identify signs of HTTP-based exfiltration in packet captures (Wireshark) and logs (Splunk), and to provide practical search queries and investigation steps.

Why it matters

- HTTP is very common; attackers hide exfiltration in the noise of legitimate web usage.
- Successful detection stops data breaches and helps trace attacker activity post-compromise.
- Organizations must detect and respond to protect sensitive data and meet compliance requirements.

How adversaries use HTTP for data exfiltration

- **POST uploads to external servers:** Bulk data is sent to attacker-controlled hosts or cloud storage in POST request bodies.
- **GET requests with encoded data:** Attacker squeezes small chunks into query strings or path segments (useful for low-and-slow exfiltration).
- **Use of common services / CDN:** Exfiltration disguised as uploads to popular services or attacker-controlled subdomains under reputable domains.
- **Custom headers:** Data placed in headers (e.g., `X-Data: <base64>`) may bypass some string-based DLP.
- **Chunked transfer / multipart:** Large payloads split into multiple requests to avoid size thresholds.
- **HTTPS/TLS tunneling:** The encrypted channel hides the payload; detection requires TLS inspection, SNI analysis, or metadata-based detection.
- **Staging via cloud services:** The attacker uploads to Dropbox/GitHub/Gist and then fetches externally.

Adversaries adapt: low-and-slow approaches, encryption/encoding, and use of legitimate services to evade detection.

Indicators of Attack (IoA) {Common network indicators}

- Unusually large HTTP POST requests to external/unexpected hosts.
- HTTP requests to domains with low reputation / rarely seen in baseline traffic.
- Frequent small requests (beaconing) to the same host, followed by large uploads.

- Chunked or multipart transfers where multiple requests compose a larger file.

Analyzing Logs in Splunk

`index="data_exfil" sourcetype="http_logs"` (Retrieves all web traffic logs from the data exfiltration index.)

`index="data_exfil" sourcetype="http_logs" method=POST` (Filters web logs specifically for outbound data uploads submitted via HTTP POST requests.)

`index="data_exfil" sourcetype="http_logs" method=POST | stats count avg(bytes_sent) max(bytes_sent) min(bytes_sent) by domain | sort - count` (Calculates request counts and byte-transfer statistics for HTTP POST uploads grouped by domain, sorted from most to least frequent.)

`index="data_exfil" sourcetype="http_logs" method=POST bytes_sent > 600 | table _time src_ip uri domain dst_ip bytes_sent | sort - bytes_sent` (Lists key details for large HTTP POST uploads exceeding 600 bytes, ordered from largest payload to smallest.)

Network Traffic Analysis Filter on HTTP traffic

`http.request.method == "POST"` (HTTP traffic with POST requests. Filter on POST requests)

`http.request.method == "POST" and frame.len > 500` (Filter on the traffic with the frame length of more than 500. **frame.len** measures the total size of a captured network packet in bytes like ethernet header, IP header, TCP/UDP header.

Answer the questions below

Which internal compromised host was used to exfiltrate this sensitive data?
192.168.1.103

`index="data_exfil" sourcetype="http_logs" method=POST bytes_sent > 600 | table _time src_ip uri domain dst_ip bytes_sent | sort - bytes_sent`

What's the flag hidden inside the exfiltrated data?

THM{http_raw_3xf1ltr4t10n_succ3ss}

Task 7 - Detection: Data Exfiltration via ICMP

ICMP is a network-layer protocol used for diagnostics and control (e.g., ping, TTL exceeded). Because it is commonly allowed through firewalls and

typically inspected less strictly than TCP/UDP, attackers sometimes abuse ICMP to tunnel and exfiltrate data. Malicious actors encode data into ICMP payloads (echo request/reply, timestamp, info) and send it to a remote listener under their control.

How adversaries use ICMP for exfiltration

Common techniques:

- ICMP echo (type 8) / reply (type 0) tunneling: attackers place encoded (base64, hex) chunks of files inside ICMP payloads. The remote server collects and decodes them.
- Custom ICMP types/codes: using uncommon ICMP types or non-zero codes to avoid signature-based detections.
- Fragmentation and reassembly: large payloads are split across multiple packets.
- Encryption/obfuscation: Encrypting or encrypting payloads (base64 is common) to look like random data.

Indicators that something may be malicious:

- Persistent ICMP sessions to an external host not used for legitimate monitoring.
- Unusually large ICMP payloads or frequent ICMP with payload > typical ping size.
- ICMP payloads that contain high-entropy data or patterns consistent with base64/hex.
- Bursts of ICMP are immediately followed by no other legitimate application traffic from the same host.

Indicators of attack in Wireshark

Look for the following in a pcap when inspecting in Wireshark:

- ICMP packet volumes: a single host sending many ICMP echo requests to an external IP.
- Large `frame.len` or `icmp.payload`: pings with payloads much larger than typical (e.g., > 64 bytes).

- ICMP type/code unusual values: e.g., unusual use of timestamp(13/14) or custom codes.
- Regular timing (periodicity): evenly spaced ICMP packets carrying similar-sized payloads.
- Fragments with reassembly: multiple ICMP fragments from the same src/dst pair..

Filter All ICMP Traffic

`icmp.type == 8` (Filters exclusively for **ICMP Echo Request** packets (outbound ping requests, assigned 8). **Purpose:** Isolates every time a host on the network attempts to reach out and check if another IP address is live.

Examine Large ICMP Packets

Let's now apply the filter on the ICMP requests and focus on the frame length over 100, as shown below:

`icmp.type == 8 and frame.len > 100` (Filters for ICMP Echo Requests whose overall network frame size **exceeds 100 bytes**. **Purpose:** Filters out standard background pings to isolate **abnormally large ping requests**. Normal pings are ~74 bytes total. Anything over 100 is suspicious.

Answer the questions below

| What is the flag found in the exfiltrated data through ICMP?

THM{1cmp_3ch0_3xf1ltr4t10n_succ3ss}

`icmp.type == 8 and frame.len > 100`

Task 8 - Conclusion

- **The Fundamentals:** What data exfiltration is and why it's a critical threat to organizations.
- **Attacker Frameworks:** How data exfiltration fits into established models like the Cyber Kill Chain (Actions on Objectives).
- **Common Techniques:** The various methods attackers use to steal data, including FTP, HTTP, DNS & ICMP tunneling.

Adversaries use more channels to exfiltrate the data, and we can look at other log sources to find indicators of the attack.